



JOBSHEET

PRAKTIKUM BASIS DATA

Jurusan Teknologi Informasi
POLITEKNIK NEGERI MALANG
2025/2026 Genap

PERTEMUAN 9

Data Definition Language (DDL)

Team Teaching:

Prof. Ir. Yan Watequlis Syaifudin, ST., M.MT., Ph.D.

Candra Bella Vista, S.Kom., MT.

Moch Zawaruddin Abdullah, S.ST., M.Kom.

Yoppy Yunhasnawa, S.ST., M.Sc.

Ridwan Rismanto, S.ST., M.Kom., Ph.D.

Elok Nur Hamdana, S.T., M.T

M. Hasyim ratsanjani, S.Kom., M.Kom.



Mata Kuliah : Praktikum Basis Data
Program Studi : D4 – Teknik Informatika / D4 – Sistem Informasi Bisnis
Semester : 4 (empat)
Pertemuan ke- : 9 (sembilan)

JOBSHEET 09

DATA DEFINITION LANGUAGE (DDL)

Tujuan Pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Membuat dan mengelola database dan tabel menggunakan perintah CREATE
2. Memodifikasi struktur tabel yang sudah ada menggunakan perintah ALTER
3. Menghapus objek database secara tepat menggunakan perintah DROP
4. Membedakan penggunaan TRUNCATE, DELETE, dan DROP dalam konteks yang sesuai
5. Menerapkan penamaan (RENAME) dan dokumentasi (COMMENT) pada objek database
6. Merancang schema database yang baik dengan constraint dan relasi yang tepat

Peralatan yang Dibutuhkan

- MariaDB Server - [download](#)
- DBeaver Community - [download](#)

Perhatian Penting

Semua perintah DDL (*CREATE*, *ALTER*, *DROP*, *TRUNCATE*) bersifat **AUTO-COMMIT** dan tidak bisa di-**ROLLBACK**. Selalu buat backup terlebih dahulu sebelum menjalankan perintah DDL pada data penting.

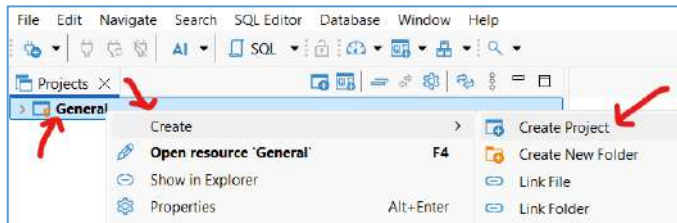
Konvensi Penulisan

HURUF BESAR → Kata kunci SQL (keyword)
huruf_kecil_garis_bawah → Nama objek yang dibuat pengguna
`-- baris komentar` → Baris komentar, tidak dieksekusi
[...] → Bagian yang bersifat opsional (boleh tidak ditulis)

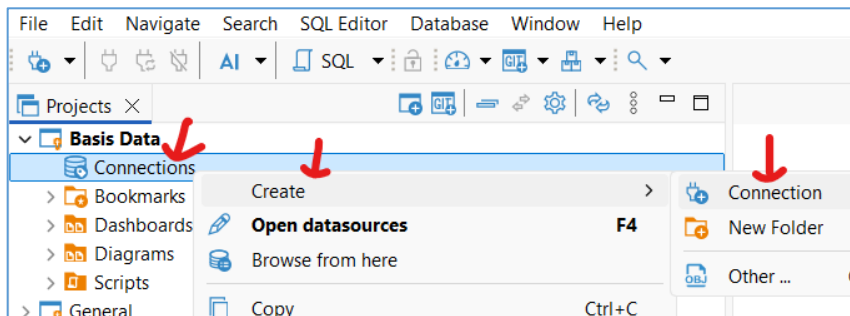
1. Setup MariaDB dan DBeaver

Sebelum memulai praktikum basis data, kita pastikan sudah melakukan instalasi MariaDB dan DBeaver. Kita ikuti Langkah-langkah berikut

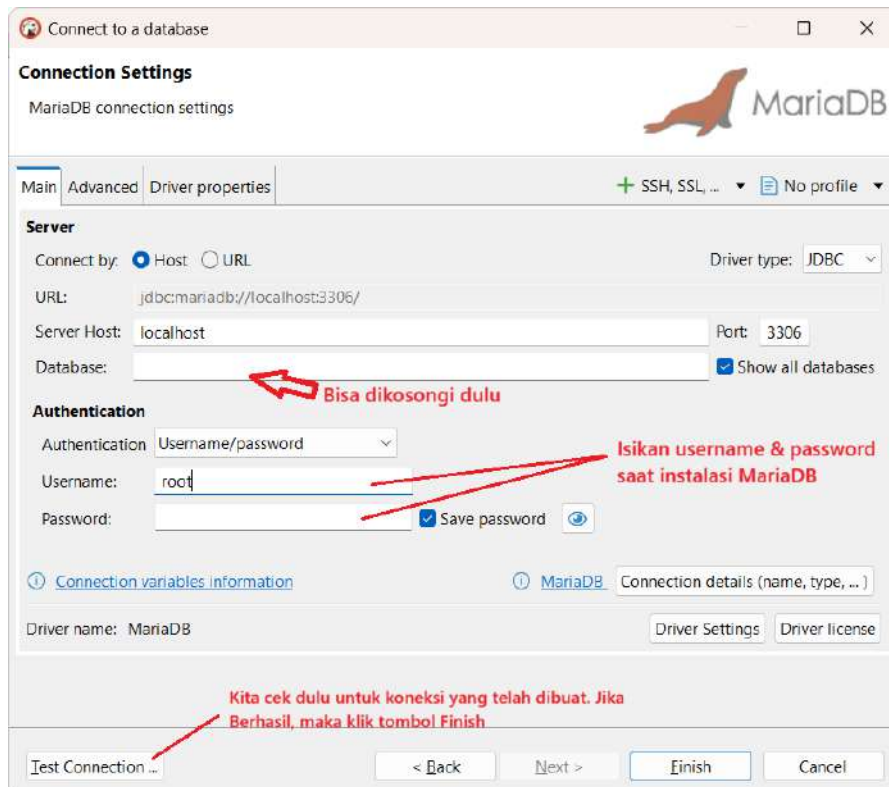
1. Buka aplikasi *DBeaver Community*
2. Kita buat project baru bernama **Basis Data**, klik menu **Window** → **Projects**
3. Pada panel Project, klik kanan **General** → **Create** → **Create Project**



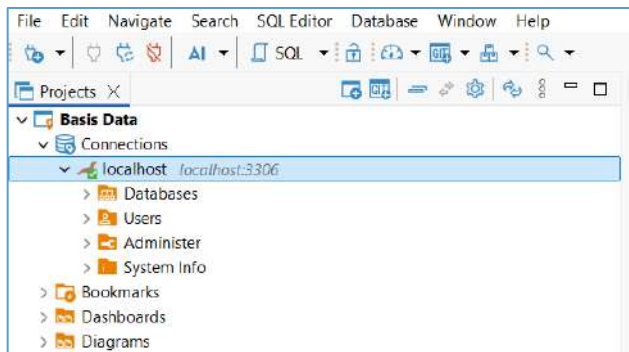
4. Ketika project Basis Data sudah berhasil dibuat, kita klik kanan project **Basis Data** → **Set Active Project**
5. Kemudian kita buat koneksi untuk DBeaver agar bisa terhubung dengan MariaDB



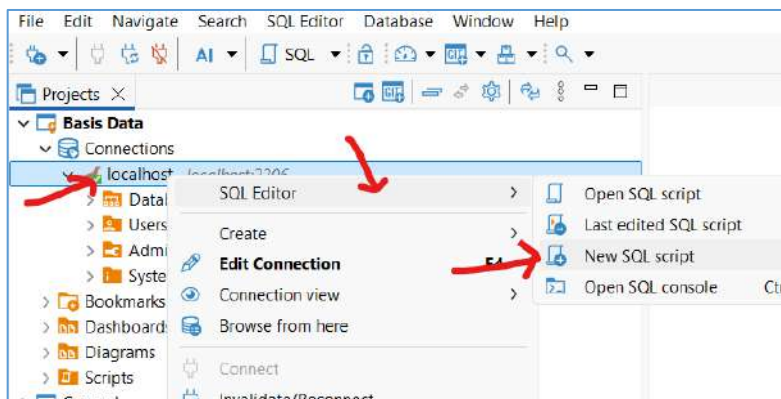
6. Kita pilih opsi basis data MariaDB → klik Next → akan muncul tampilan



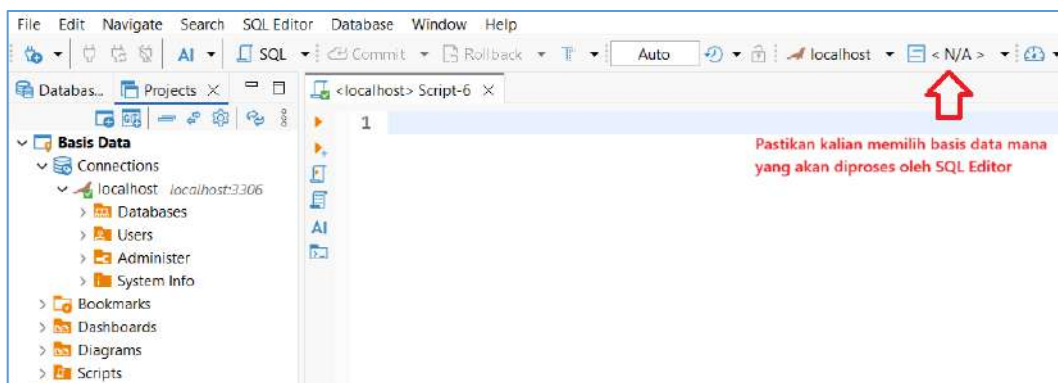
7. Ketika berhasil melakukan koneksi ke MariaDB, maka tampilan akan seperti ini



8. Selanjutnya kita siapkan SQL Editor untuk melakukan praktikum basis data ini



9. Pastikan nanti untuk memilih database yang akan diproses



10. Persiapan selesai.

2. **CREATE** – Membuat Objek pada Database

Perintah **CREATE** adalah perintah DDL yang digunakan untuk membuat objek baru di dalam database. Objek yang dapat dibuat meliputi *database/skema*, *tabel*, *index*, *view*, dan objek lain.

Kategori SQL dan Posisi DDL

Kategori	Kepanjangan	Perintah Utama	Sifat
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Auto-commit
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Bisa ROLLBACK
DQL	Data Query Language	SELECT	Bisa ROLLBACK
DCL	Data Control Language	GRANT, REVOKE	Auto-commit
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Kontrol transaksi

2.1 Perintah CREATE DATABASE

CREATE DATABASE merupakan perintah untuk membuat container utama tempat semua objek database disimpan. Setiap database memiliki pengaturan **charset** dan **collation** yang menentukan cara penyimpanan dan perbandingan karakter.

Sintaks

```
-- Membuat database baru dengan charset default
CREATE DATABASE <database-name>;

-- Membuat database baru dengan charset yang telah ditentukan
CREATE DATABASE <database-name>
    CHARACTER SET <charset>
    COLLATE <collation>;

-- Buat database baru jika belum ada database dengan nama yang sama
CREATE DATABASE IF NOT EXISTS <database-name>;
```

Character Set (Charset)

Charset adalah standar pengkodean karakter yang merupakan aturan tentang bagaimana sebuah karakter (huruf, angka, simbol) direpresentasikan sebagai angka biner di dalam komputer dan disimpan ke disk.

Analoginya seperti kamus: charset mendefinisikan "huruf A disimpan sebagai angka 65", "huruf B sebagai 66", dan seterusnya. Setiap charset memiliki daftar karakter yang berbeda-beda dan kapasitas yang berbeda.

Contoh charset yang umum di MySQL:

Charset	Karakter yang didukung	Byte per karakter
ascii	Huruf Latin dasar + angka + simbol	1 byte
latin1	Latin + karakter Eropa Barat	1 byte
utf8	Unicode 3 byte (tidak lengkap, tanpa emoji)	1-3 byte
utf8mb4	Unicode penuh, termasuk emoji dan semua bahasa	1-4 byte

Collation

Collation adalah aturan tentang cara membandingkan dan mengurutkan karakter. Jika *charset* menjawab pertanyaan "bagaimana karakter disimpan?", maka *collation* menjawab "bagaimana karakter dibandingkan?".

Collation menentukan hal-hal seperti apakah “A” = “a” dalam pencarian, apakah “é” = “e”, atau bagaimana urutan “ä”, “ö”, “ü” dalam pengurutan bahasa Jerman vs Swedia. Penamaan *collation* mengikuti pola `charset_bahasa_sensitivitas`, contoh:

Collation	Arti
<code>utf8mb4_unicode_ci</code>	utf8mb4, aturan Unicode, case-insensitive
<code>utf8mb4_unicode_cs</code>	utf8mb4, aturan Unicode, case-sensitive
<code>utf8mb4_bin</code>	Perbandingan biner murni (paling ketat)
<code>utf8mb4_general_ci</code>	utf8mb4, aturan umum (lebih cepat, kurang akurat)

Akhiran yang perlu diingat: `*_ci` = *case-insensitive*, `*_cs` = *case-sensitive*,
`*_bin` = *binary*.

➤ Praktikum 1

Pada praktikum 1 ini, kita akan mencoba membuat container database. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
1  -- buat container database dengan nama basis_data_1
2  CREATE DATABASE basis_data_1;
3
4  -- gunakan container basis_data_1
5  USE basis_data_1;
6
7  -- tampilkan detail pembuatan container basis_data_1
8  SHOW CREATE DATABASE basis_data_1;
```

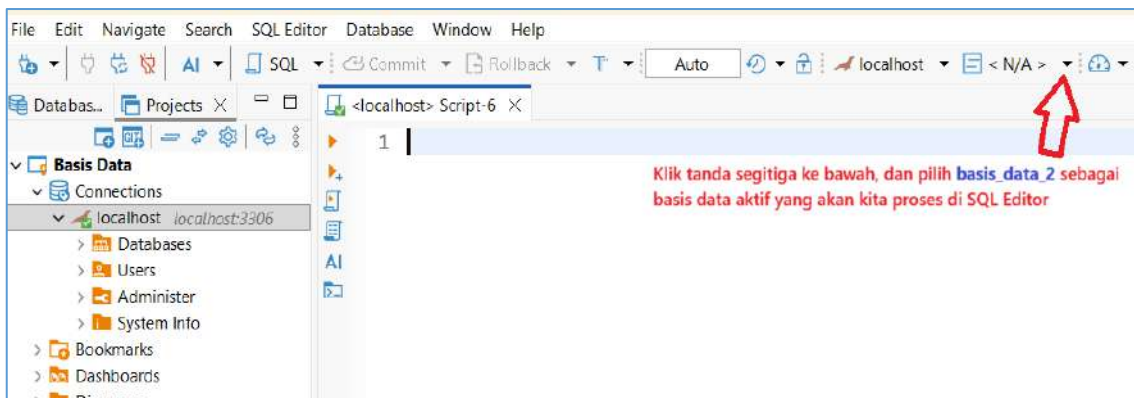
2. Eksekusi perintah pada baris ke-2 dengan cara → arahkan cursor ke baris ke-2, kemudian pencet tombol **CTRL+ENTER**. Amati dan jelaskan apa hasil dari eksekusi
3. Eksekusi perintah pada baris ke-5 dengan cara → arahkan cursor ke baris ke-5, kemudian pencet tombol **CTRL+ENTER**. Amati dan jelaskan apa hasil dari eksekusi
4. Eksekusi perintah pada baris ke-8 dengan cara → arahkan cursor ke baris ke-8, kemudian pencet **CTRL+ENTER**. Amati dan jelaskan apa hasil dari eksekusi
5. Sekarang, coba praktikkan juga untuk kode SQL di bawah ini

```

1  -- buat container database dengan nama basis_data_2
2  CREATE DATABASE basis_data_2
3      CHARACTER SET utf8mb4
4      COLLATE utf8mb4_unicode_ci;
5
6  -- gunakan container basis_data_2
7  USE basis_data_2;
8
9  -- tampilkan detail pembuatan container basis_data_2
10 SHOW CREATE DATABASE basis_data_2;

```

6. Eksekusi untuk masing-masing kode SQL, dan amati apa perbedaan antara kode SQL pada Langkah 1 dengan Langkah 5?
7. Selanjutnya, kita fokuskan ke penggunaan **basis_data_2** untuk melakukan praktikum selanjutnya
8. Kita set database yang aktif pada SQL Editor di DBeaver, agar kode perintah SQL yang kita eksekusi, akan masuk semua di basis_data_2



2.2 Perintah CREATE TABLE

CREATE TABLE mendefinisikan struktur tabel — nama kolom, tipe data, *constraint*, *index*, dan *engine* penyimpanan. Ini adalah perintah DDL yang paling sering digunakan.

Sintaks

```

-- Membuat tabel baru
CREATE TABLE <table-name> (
    <column-name> <DATA-TYPE> [CONSTRAINT] ,
    <column-name> <DATA-TYPE> [CONSTRAINT] ,
    <column-name> <DATA-TYPE> [CONSTRAINT] ,
    ...
    [table-constraint-definition]
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Type Data Penting MySQL

Kategori	Type Data	Kegunaan
Numerik	INT / BIGINT UNSIGNED	ID, counter, jumlah stok
	DECIMAL(p,s)	Harga, saldo, gaji — presisi tinggi
	FLOAT / DOUBLE	Koordinat, statistik (tidak presisi)
	BOOLEAN / TINYINT(1)	Flag benar/salah
Teks	CHAR(n)	Teks panjang tetap: kode_pos, iso_negara
	VARCHAR(n)	Teks dinamis: nama, email, alamat
	TEXT / LONGTEXT	Teks panjang: deskripsi, artikel
	ENUM('a', 'b', ...)	Nilai terbatas: status, jenis, peran
Tanggal	DATE	Tanggal saja: tgl_lahir, tgl_expired
	DATETIME	Tanggal + waktu: jadwal, reservasi
	TIMESTAMP	created_at, updated_at (timezone-aware)

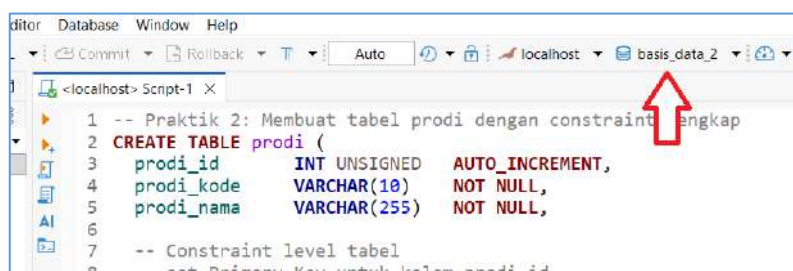
Constraint Kolom pada Tabel Basis Data

Constraint	Fungsi	NULL?	Duplikat?
NOT NULL	Kolom wajib diisi	Tidak	Boleh
NULL	Kolom boleh kosong (default)	Boleh	Boleh
DEFAULT <nilai>	Nilai otomatis jika tidak diisi	Opsional	Boleh
AUTO_INCREMENT	Angka bertambah otomatis (PK)	Tidak	Tidak
UNIQUE	Nilai harus unik dalam kolom	1x saja	Tidak
PRIMARY KEY	Identitas unik baris (NOT NULL+UNIQUE)	Tidak	Tidak
CHECK (kondisi)	Validasi nilai — MySQL 8.0.16+	Opsional	Boleh
FOREIGN KEY	Relasi ke tabel lain (InnoDB only)	Opsional	Boleh

➤ Praktikum 2

Pada praktikum 2 ini, kita akan mencoba membuat table pada basis data. Langkah-langkah praktikum:

1. Buka SQL Editor, pastikan database aktifnya adalah basis_data_2



2. ketikkan perintah berikut untuk membuat table **Prodi**

```
1 -- Praktik 2: Membuat tabel prodi dengan constraint lengkap
2 CREATE TABLE prodi (
3     prodi_id      INT UNSIGNED    AUTO_INCREMENT,
4     prodi_kode    VARCHAR(10)    NOT NULL,
5     prodi_nama    VARCHAR(255)   NOT NULL,
6
7     -- Constraint level tabel
8     -- set Primary Key untuk kolom prodi_id
9     PRIMARY KEY   (prodi_id),
10
11    -- set Unique Key untuk kolom prodi_kode
12    UNIQUE KEY     unique_prodi (prodi_kode)
13 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

3. Eksekusi perintah SQL dengan cara block kode SQL pada awal baris ke-2 sampai dengan akhir baris 13 → kemudian pencet tombol **CTRL+ENTER**. Amati dan jelaskan apa hasil dari eksekusi
4. Kemudian coba eksekusi perintah SQL berikut dan amati apa yang terjadi

```
DESCRIBE prodi;
```

5. Selanjutnya kita coba membuat table lain yaitu table **mahasiswa**

```
1 -- Praktik 2: Membuat tabel mahasiswa dengan constraint lengkap
2 CREATE TABLE mahasiswa (
3     mahasiswa_id  INT UNSIGNED    AUTO_INCREMENT,
4     nim           CHAR(10)        NOT NULL,
5     nama_lengkap  VARCHAR(150)   NOT NULL,
6     email         VARCHAR(255)   NOT NULL,
7
8     -- Kolom akademik
9     prodi_id      INT UNSIGNED    NOT NULL,
10    angkatan       YEAR            NOT NULL,
11    ipk            DECIMAL(3,2)    DEFAULT 0.00,
12    status         ENUM('aktif','cuti','lulus','do') NOT NULL DEFAULT 'aktif',
13
14    -- Constraint level tabel
15    -- set Primary Key untuk kolom mahasiswa_id
16    PRIMARY KEY    (mahasiswa_id),
17
18    -- set Unique Key untuk kolom NIM
19    UNIQUE KEY     unique_nim (nim),
20
21    -- set Index untuk kolom yang akan dijadikan Foreign Key
22    INDEX          idx_prodi (prodi_id),
23
24    -- set Foreign Key untuk kolom prodi_id me-refer ke tabel Prodi
25    CONSTRAINT     fk_mahasiswa_prodi
26        FOREIGN KEY (prodi_id) REFERENCES prodi(prodi_id)
27        ON DELETE RESTRICT
28        ON UPDATE CASCADE
29 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

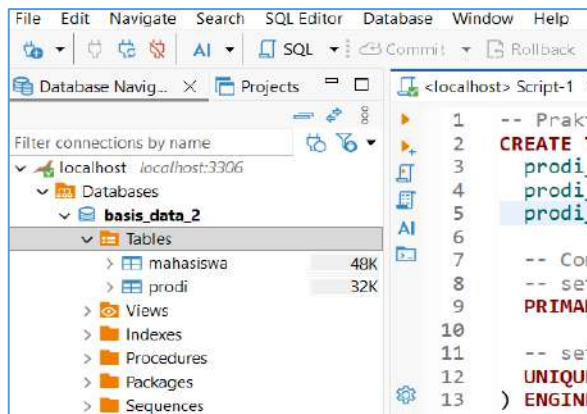
6. Coba kita amati table **mahasiswa** yang barusan kita buat

```
DESCRIBE mahasiswa;
```

7. Untuk melihat table-table yang telah kita buat, kita bisa menggunakan perintah

```
SHOW TABLES;
```

atau kita bisa melihat (me-refresh) tampilan dari DBeaver dengan cara Pilih **basis_data_2** → klik kanan → pilih refresh → expand menu Tables



2.3 Perintah **CREATE INDEX**

INDEX mempercepat operasi **SELECT** dengan cara membuat struktur data tambahan (B-Tree). Tanpa index, MySQL melakukan full table scan, membaca setiap baris satu per satu.

INDEX bisa di buat saat membuat sebuah table (sudah dicontohkan di **Praktikum 2**) ataupun bisa dibuat secara terpisah.

Sintaks

```
-- Membuat index baru secara terpisah
CREATE [OPTION-TYPE] INDEX <index-description>;
```

[*OPTION-TYPE*] dapat berupa beberapa jenis index diantaranya:

- KOSONG (tidak diisi) → jenis index standard yang digunakan secara umum
- UNIQUE** → jenis index yang menjamin isi data dalam kolom tersebut bernilai unik (tidak ada yang sama). Akan tetapi index ini bisa bernilai NULL.
- FULLTEXT** → jenis index yang dirancang untuk pencarian teks panjang.

➤ **Praktikum 3**

Pada praktikum 3 ini, kita akan mencoba membuat index untuk table **mahasiswa**. Langkah-langkah praktikum:

- Buka SQL Editor, ketikkan perintah berikut untuk membuat index

```
1  -- Praktik 3: Membuat berbagai jenis index
2
3  -- Index biasa (nilai kolom boleh duplikat)
4  CREATE INDEX index_nama_mhs ON mahasiswa (nama_lengkap);
5
6  -- Index unik (nilai harus unik)
7  CREATE UNIQUE INDEX index_unique_nim ON mahasiswa (nim);
8
9  -- Composite index (bisa melakukan index lebih dari satu kolom)
10 CREATE INDEX idx_status_angkatan ON mahasiswa (status, angkatan);
11
12 -- Full-text index (pencarian teks bebas)
13 CREATE FULLTEXT INDEX index_fulltext_nama ON mahasiswa (nama_lengkap);
```

2. Eksekusi perintah SQL secara satu per satu
3. Kita bisa melihat daftar index yang kita buat pada table mahasiswa dengan menggunakan perintah

```
-- Lihat semua index pada tabel  
SHOW INDEX FROM mahasiswa;
```

4. Amati dan catat apa yang kamu ketahui

2.4 Perintah **CREATE VIEW**

VIEW adalah tabel virtual yang dibentuk dari hasil query **SELECT** (ada dibagian pembahasan *DQL (Data Query Language)* atau *DRL (Data Retrieval Language)*). Data tidak disimpan secara fisik pada sebuah table, setiap kali view diakses, query dasarnya dieksekusi ulang. Berguna untuk menyederhanakan query kompleks dan membatasi akses kolom sensitif.

Sintaks

```
-- Membuat view  
CREATE VIEW <view-description>;
```

➤ **Praktikum 4**

Pada praktikum 4 ini, kita akan mencoba membuat view dari table **mahasiswa** dan **prodi**.

Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan dan eksekusi perintah berikut untuk membuat view

```
1 CREATE VIEW vw_mahasiswa_prodi AS  
2 SELECT m.mahasiswa_id, m.nim, m.nama_lengkap, m.email,  
3 p.prodi_id, p.prodi_kode, p.prodi_nama,  
4 m.angkatan, m.ipk, m.status  
5 FROM mahasiswa m  
6 JOIN prodi p ON p.prodi_id = m.prodi_id;
```

2. Selanjutnya kita bisa melihat daftar table ataupun view yang telah kita buat dengan mengeksekusi perintah berikut

```
-- Perintah untuk lihat semua tabel beserta jenisnya  
SHOW FULL TABLES;  
  
-- perintah untuk cek deskripsi kolom pada tabel  
DESCRIBE vw_mahasiswa_prodi;
```

3. Amati dan catat apa yang kamu pahami

3. **ALTER TABLE** – Memodifikasi Struktur

ALTER TABLE digunakan untuk memodifikasi struktur tabel yang sudah ada tanpa harus menghapus dan membuat ulang tabel tersebut. Semua operasi ALTER bersifat *auto-commit*

yang artinya segala perubahan struktur tabel yang dilakukan langsung disimpan secara permanen ke dalam database saat itu juga, perubahan struktur tabel tersebut tidak bisa dibatalkan atau dikembalikan (*rollback*).

3.1 Perintah ADD COLUMN

ADD COLUMN merupakan perintah untuk menambahkan kolom baru pada sebuah table basis data.

Sintaks

```
-- Membuat kolom baru secara default (di posisi akhir)
ALTER TABLE <table-name> ADD COLUMN <column-name> <data-type>;

-- Membuat kolom baru setelah kolom tertentu
ALTER TABLE <table-name> ADD COLUMN <column-name> <data-type> AFTER <other-column>;

-- Membuat kolom baru di posisi pertama
ALTER TABLE <table-name> ADD COLUMN <column-name> <data-type> FIRST;

-- Membuat beberapa kolom sekaligus
ALTER TABLE <table-name>
    ADD COLUMN <column-name> <data-type>,
    ADD COLUMN <column-name> <data-type>,
    ADD COLUMN <column-name> <data-type>;
```

➤ Praktikum 5

Pada praktikum 5 ini, kita akan mencoba membuat kolom baru pada table **Prodi**. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
1  -- Membuat kolom baru secara default (di posisi akhir)
2  ALTER TABLE prodi ADD COLUMN prodi_jenang VARCHAR(10);
3
4  -- Membuat kolom baru setelah kolom tertentu
5  ALTER TABLE prodi ADD COLUMN jurusan VARCHAR(50) AFTER prodi_id;
6
7  -- Membuat kolom baru di posisi pertama
8  ALTER TABLE prodi ADD COLUMN prodi_alias VARCHAR(10) FIRST;
9
10 -- Membuat beberapa kolom sekaligus
11 ALTER TABLE prodi
12     ADD COLUMN ketua_prodi VARCHAR(50),
13     ADD COLUMN tahun_berdiri YEAR,
14     ADD COLUMN akreditasi enum('Unggul', 'Sangat Baik', 'Baik');
```

2. Eksekusi perintah SQL satu per satu
3. Gunakan perintah berikut untuk melihat hasil akhir table prodi

```
DESCRIBE prodi;
```

4. Amati dan catat apa yang terjadi

3.2 Perintah **MODIFY COLUMN** dan **CHANGE COLUMN**

MODIFY COLUMN digunakan untuk **mengubah tipe data** atau constraint kolom yang sudah ada (nama kolom tetap). Sedangkan **CHANGE COLUMN** digunakan untuk mengubah **nama kolom sekaligus tipe datanya**.

Sintaks

```
-- Modify : hanya mengubah tipe data kolom
ALTER TABLE <table-name> MODIFY COLUMN <column-name> <new-data-type>;

-- Change : mengubah nama kolom dan tipe data nya
ALTER TABLE <table-name> CHANGE COLUMN <old-column-name> <new-column-name>
<new-data-type>;
```

➤ **Praktikum 6**

Pada praktikum 6 ini, kita akan mencoba memodifikasi beberapa kolom pada table **mahasiswa** dan **prodi**. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
1 -- Modify : hanya mengubah tipe data kolom tabel mahasiswa
2 ALTER TABLE mahasiswa MODIFY COLUMN nim VARCHAR(15);
3
4 -- Change : mengubah nama kolom dan tipe data nya untuk tabel prodi
5 ALTER TABLE prodi CHANGE COLUMN ketua_prodi koordinator_prodi VARCHAR(100);
```

2. Eksekusi perintah SQL satu per satu
3. Gunakan perintah **DESCRIBE <table-name>** untuk melihat perubahan yang terjadi
4. Amati dan jelaskan pendapatmu

3.3 Perintah **RENAME COLUMN**

RENAME COLUMN digunakan untuk **mengubah nama kolom** tanpa mengganti tipe datanya. Hal ini berbanding terbalik dengan **MODIFY COLUMN** yang hanya mengganti tipe data tanpa mengubah nama kolom.

Sintaks

```
-- Modify : hanya mengubah tipe data kolom
ALTER TABLE <table-name> RENAME COLUMN <old-column-name> TO <new-column-name>;
```

Catatan perintah **RENAME COLUMN** hanya tersedia untuk basis data versi

- ✓ **MySQL 8.0** ~ terbaru
- ✓ **MariaDB 10.5.2** ~ terbaru

Jika versi basis data berada di bawahnya, maka perintah tersebut tidak bisa dijalankan.

➤ Praktikum 7

Pada praktikum 7 ini, kita akan mencoba mengubah nama kolom pada table **mahasiswa**.

Langkah-langkah praktikum:

5. Buka SQL Editor, ketikkan perintah berikut

```
1  -- Rename 1 kolom
2  ALTER TABLE mahasiswa RENAME COLUMN nim TO mahasiswa_nim;
3
4  -- Rename beberapa kolom sekaligus
5  ALTER TABLE prodi
6      RENAME COLUMN nama_lengkap TO mahasiswa_nama,
7      RENAME COLUMN email TO mahasiswa_email;
```

6. Eksekusi perintah SQL satu per satu
7. Gunakan perintah **DESCRIBE** *<table-name>* untuk melihat perubahan yang terjadi
8. Amati dan jelaskan pendapatmu

3.4 Perintah **DROP COLUMN**, **DROP INDEX**, dan **DROP FOREIGN KEY**

Perintah **DROP** digunakan untuk menghapus objek pada sebuah table. Terdapat beberapa jenis perintah DROP pada ALTER TABLE, diantaranya:

- **DROP COLUMN** merupakan perintah yang digunakan untuk **menghapus kolom beserta datanya** pada sebuah table.
- **DROP INDEX** merupakan perintah untuk menghapus index yang ada pada sebuah table
- **DROP FOREIGN KEY** Adalah perintah yang digunakan untuk menghapus relasi foreign key saja, tanpa menghapus kolom dan datanya.

Sintaks

```
-- menghapus satu kolom
ALTER TABLE <table-name> DROP COLUMN <column-name>;

-- menghapus beberapa kolom sekaligus
ALTER TABLE <table-name>
    DROP COLUMN <column-name>,
    DROP COLUMN <column-name>,
    DROP COLUMN <column-name>;

-- menghapus satu index
ALTER TABLE <table-name> DROP INDEX <index-name>;

-- menghapus beberapa index sekaligus
ALTER TABLE <table-name>
    DROP INDEX <index-name>,
    DROP INDEX <index-name>,
    DROP INDEX <index-name>;

-- menghapus satu relasi foreign key
ALTER TABLE <table-name> DROP FOREIGN KEY <fk-name>;

-- menghapus beberapa foreign key sekaligus
ALTER TABLE <table-name>
```

```
DROP FOREIGN KEY <fk-name>,  
DROP FOREIGN KEY <fk-name>,  
DROP FOREIGN KEY <fk-name>;
```

➤ Praktikum 8

Pada praktikum 8 ini, kita akan mencoba mengubah nama kolom pada table **mahasiswa** dan **prodi**. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
1  -- menghapus satu kolom pada tabel prodi  
2  ALTER TABLE prodi DROP COLUMN prodi_alias;  
3  
4  -- Lihat semua index pada tabel mahasiswa  
5  SHOW INDEX FROM mahasiswa;  
6  
7  -- menghapus satu index  
8  ALTER TABLE mahasiswa DROP INDEX index_unique_nim;  
9  
10 -- menghapus satu relasi foreign key  
11 ALTER TABLE mahasiswa DROP FOREIGN KEY fk_mahasiswa_prodi;  
12  
13 -- cek kolom tabel mahasiswa setelah FK dihapus  
14 DESCRIBE mahasiswa;
```

2. Eksekusi perintah SQL satu per satu
3. Gunakan perintah **DESCRIBE <table-name>** untuk melihat perubahan yang terjadi
4. Amati dan jelaskan pendapatmu

3.5 Perintah **RENAME TO**

Perintah **RENAME TO** digunakan untuk menubah nama sebuah table.

Sintaks

```
-- menghapus satu kolom  
ALTER TABLE <table-name> RENAME TO <new-table-name>;
```

➤ Praktikum 9

Pada praktikum 9 ini, kita akan mencoba mengubah nama table **mahasiswa** dan **prodi**. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan dan eksekusi perintah berikut

```
1  -- Rename tabel prodi menjadi program_studi  
2  ALTER TABLE prodi RENAME TO program_studi;
```

2. Gunakan perintah **SHOW TABLES** untuk melihat perubahan yang terjadi
3. Amati dan jelaskan pendapatmu

4. DROP – Menghapus Objek pada Database

Perintah **DROP** adalah perintah DDL yang digunakan untuk menghapus objek yang ada di dalam database. Objek yang dapat dibuat meliputi *database/skema*, *tabel*, *index*, *view*, dan objek lain.

4.1 Perintah DROP INDEX

DROP INDEX merupakan perintah untuk menghapus objek index yang ada pada sebuah tabel. INDEX bisa berupa index PRIMARY KEY, INDEX (normal index), UNIQUE INDEX, FULLTEXT INDEX, dan FOREIGN KEY INDEX.

Sintaks

```
-- Membuat database baru dengan charset default
DROP INDEX <index-name> ON <table-name>;

-- DROP INDEX merupakan alternatif dari
ALTER TABLE <table-name> DROP INDEX <index-name>;
```

➤ Praktikum 10

Pada praktikum 10 ini, kita akan mencoba menghapus index pada table **mahasiswa**. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
1  -- cek semua index yang ada di tabel mahasiswa
2  SHOW INDEX FROM mahasiswa;
3
4  DROP INDEX index nama mhs ON mahasiswa;
```

2. Eksekusi perintah satu per satu
3. Amati dan jelaskan pendapatmu

4.2 Perintah DROP VIEW

DROP VIEW digunakan untuk menghapus view atau table virtual yang ada di database.

Sintaks

```
-- menghapus satu view
DROP VIEW <view-name>;

-- Gunakan IF EXISTS sebelum menghapus,
-- agar menghindari error saat view tidak ditemukan
DROP VIEW IF EXISTS <view-name>;

-- menghapus beberapa view sekaligus
DROP VIEW IF EXISTS
    <view-name>,
    <view-name>,
    <view-name>;
```

➤ Praktikum 11

Pada praktikum 11 ini, kita akan mencoba menghapus view yang sudah pernah dibuat. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
SHOW FULL TABLES;
```

Maka akan muncul hasil daftar table dan view

	AZ Tables_in_basis_data_1	AZ Table_type
1	mahasiswa	BASE TABLE
2	program_studi	BASE TABLE
3	vw_mahasiswa_prodi	VIEW

2. Kemudian kita jalankan perintah berikut untuk menghapus view

```
DROP VIEW vw_mahasiswa_prodi;
```

3. Lakukan Kembali Langkah nomor 1, amati dan jelaskan apa yang terjadi.

4.3 Perintah DROP TABLE

DROP TABLE digunakan untuk menghapus table beserta data yang ada di dalamnya. Perlu diperhatikan jika ingin menghapus table. Tabel yang ingin dihapus **HARUS TERBEBAS** dari relasi constraint yang melekat padanya (*yang artinya, primary key dari table yang ingin dihapus, tidak menjadi dijadikan foreign key oleh table lain, atau ada view yang me-refer table yang akan dihapus*).

Sintaks

```
-- menghapus tabel  
DROP TABLE <table-name>;
```

Teknik aman sebelum melakukan DROP TABLE

Langkah	Aksi	Tujuan
1	Backup data (mysqldump)	Pemulihan jika terjadi kesalahan
2	Cek nama objek dengan tepat	Hindari typo yang berakibat fatal
3	Cek dependensi (FK, view)	Pastikan tidak ada objek lain yang bergantung
4	Coba di staging terlebih dahulu	Validasi script sebelum production
5	Gunakan IF EXISTS	Script aman dijalankan ulang
6	Pertimbangkan RENAME dulu	Beri waktu validasi sebelum hapus permanen
7	Jalankan saat traffic rendah	Minimalkan dampak table lock

➤ Praktikum 12

Pada praktikum 12 ini, kita akan mencoba menghapus table **prodi**. Langkah-langkah praktikum:

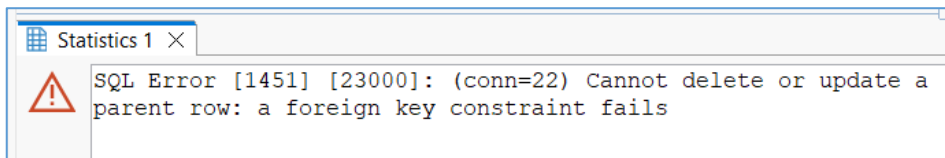
1. Buka SQL Editor, ketikkan dan eksekusi perintah berikut untuk membuat **Foreign Key** baru di table **mahasiswa** (karena **Foreign Key** lama sudah di hapus)

```
1 -- Kita buat Foreign Key baru untuk tabel mahasiswa
2 -- mengacu ke tabel Prodi
3 ALTER TABLE mahasiswa ADD CONSTRAINT fk_mahasiswa_program_studi
4 FOREIGN KEY (prodi_id)
5 REFERENCES program_studi (prodi_id);
```

2. Kemudian kita coba eksekusi perintah hapus table **program_studi**

```
-- kemudian kita coba hapus tabel program_studi
DROP TABLE program_studi;
```

3. Maka akan muncul error, karena table **program_studi** masih di refer oleh table lain yaitu table **mahasiswa**



4. Sekarang coba kalian hapus Foreign Key yang ada pada table **mahasiswa** menggunakan ALTER TABLE – DROP FOREIGN KEY
5. Kemudian coba hapus lagi table **program_studi**
6. Amati dan jelaskan pendapatmu

5. TRUNCATE TABLE, RENAME TABLE, dan COMMENT .

5.1 Perintah TRUNCATE TABLE

TRUNCATE TABLE mengosongkan seluruh data dalam tabel secara cepat sekaligus mereset nilai AUTO_INCREMENT ke 1. Struktur tabel yang telah dilakukan truncate akan tetap utuh.

Sintaks

```
-- mengosongkan isi data dalam sebuah tabel
TRUNCATE TABLE <table-name>;
```

Perbandingan TRUNCATE vs DELETE vs DROP

Aspek	TRUNCATE TABLE	DELETE FROM	DROP TABLE
Kategori SQL	DDL	DML	DDL
Hapus struktur	Tidak	Tidak	Ya
Hapus data	Semua baris	Tergantung WHERE	Semua (bersama struktur)

Klausu WHERE	Tidak bisa	Bisa	Tidak ada
Bisa ROLLBACK	Tidak	Ya (dalam transaksi)	Tidak
Reset AUTO_INCREMENT	Ya, ke 1	Tidak	Ya
Aktifkan TRIGGER	Tidak	Ya	Tidak
Kecepatan	Sangat cepat	Lambat (row-by-row)	Sangat cepat

➤ Praktikum 13

Pada praktikum 13 ini, kita akan mencoba mengosongkan data pada table **mahasiswa**. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
-- mengosongkan isi data tabel mahasiswa
TRUNCATE TABLE mahasiswa;
```

2. Saat eksekusi perintah tersebut, data di table mahasiswa akan dihapus semua (walau untuk saat ini belum ada data pada table tersebut)
3. Amati dan jelaskan pendapatmu

5.2 Perintah RENAME TABLE

RENAME TABLE merupakan perintah untuk mengubah nama sebuah table. Perintah ini merupakan alternatif dari perintah **ALTER TABLE <table-name> RENAME TO <new-table-name>**.

Sintaks

```
-- mengubah nama satu tabel
RENAME TABLE <old-table-name> TO <new-table-name>;

-- mengubah beberapa nama tabel sekaligus
RENAME TABLE
    <old-table-name> TO <new-table-name>,
    <old-table-name> TO <new-table-name>,
    <old-table-name> TO <new-table-name>;
```

➤ Praktikum 14

Pada praktikum 14 ini, kita akan mencoba mengubah nama table **mahasiswa** menjadi **m_mahasiswa**. Langkah-langkah praktikum:

1. Buka SQL Editor, ketikkan perintah berikut

```
RENAME TABLE
    mahasiswa TO m_mahasiswa,
    program_studi TO m_prodi;
```

2. Jalankan perintah tersebut. Amati dan jelaskan pendapatmu

5.3 COMMENT

COMMENT berfungsi sebagai dokumentasi yang disimpan langsung di dalam schema database. Sangat berguna untuk menjelaskan tujuan kolom, format data, atau aturan bisnis kepada developer lain..

Sintaks

```
-- Komen pada baris tunggal

/*
 * Untuk komen yang
 * membutuhkan baris lebih dari 1
 */

-- Komen deskripsi pada tabel
ALTER TABLE <table-name> COMMENT = '<new-table-name>';
```

Contoh

```
-- Tambah atau perbarui comment tabel
ALTER TABLE mahasiswa
    COMMENT = 'Data mahasiswa – schema v2.0. Diperbarui 2024-01-15.';

ALTER TABLE nilai
    COMMENT = 'Nilai akhir per MK per semester. Source of truth untuk transkrip.';

ALTER TABLE tb_session
    COMMENT = 'Session login aktif. TTL 24 jam. TRUNCATE aman dilakukan kapan saja.';
```

```
CREATE TABLE mahasiswa (
    id_mahasiswa INT UNSIGNED AUTO_INCREMENT,
    nim          CHAR(10) NOT NULL,
    nama_lengkap VARCHAR(150) NOT NULL,
    PRIMARY KEY (id_mahasiswa)
) ENGINE=InnoDB
COMMENT='Data mahasiswa aktif. Sync dari SIAK tiap malam pukul 00.00.';
```

```

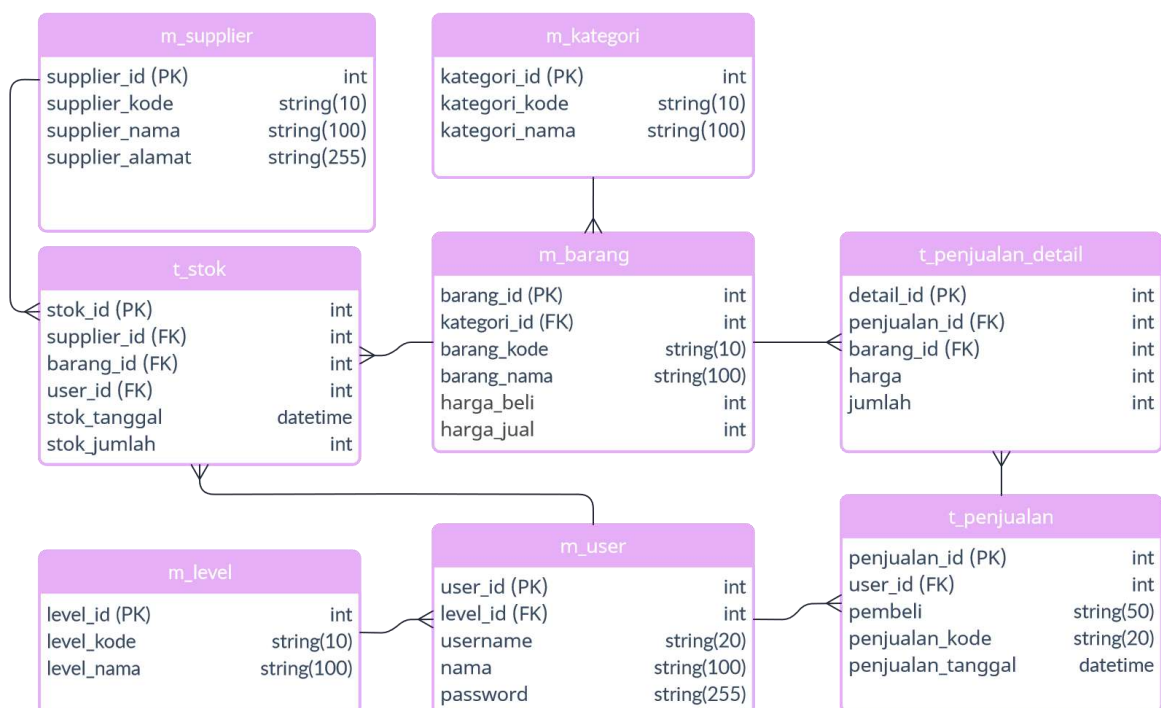
CREATE TABLE mahasiswa (
  id_mahasiswa INT UNSIGNED AUTO_INCREMENT COMMENT 'PK – identitas unik mahasiswa',
  nim CHAR(10) NOT NULL COMMENT 'Nomor Induk Mahasiswa, format: YYYNNNNNNN',
  nama_lengkap VARCHAR(150) NOT NULL COMMENT 'Nama sesuai KTP, kapital di tiap kata',
  ipk DECIMAL(3,2) DEFAULT 0.00 COMMENT 'Indeks Prestasi Kumulatif',
  status ENUM('aktif','cuti','lulus','do')
    DEFAULT 'aktif'
    COMMENT 'do = drop out. Diperbarui setiap akhir semester',
  id_prodi INT UNSIGNED NOT NULL
    COMMENT 'FK ke tabel program_studi.id_prodi',
  dibuat_pada TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    COMMENT 'Waktu data pertama kali dibuat, auto-set oleh DB',

  PRIMARY KEY (id_mahasiswa)
) ENGINE=InnoDB;

```

6. Tugas

1. Buatlah DDL berdasarkan desain PDM di bawah ini.
2. Buat laporan mengenai tugas yang dikerjakan
3. Laporan bisa menggunakan template laporan Jobsheet yang sudah disediakan



-- Selamat Mengerjakan --